



APP MÉTIER

GESTION DES ENQUÊTES PÉNALES

Plateforme de gestion intégrée des enquêtes et dossiers d'instruction pour un parquet de tribunal judiciaire — multi-contentieux, multi-utilisateurs, avec synchronisation, alertes et cartographie relationnelle.

Auteur : Audran Chevalier

Contexte : Parquet du Tribunal Judiciaire d'Amiens

Version : 1.0 · **Document :** Présentation générale

Sommaire

1 En bref

2 Contexte et public visé

3 Architecture en un coup d'œil

4 Tour des fonctionnalités

4.1 — Tableau de bord et navigation multi-contentieux

4.2 — Enquêtes : cycle de vie complet

4.3 — Instructions judiciaires

4.4 — Cartographie relationnelle

4.5 — Statistiques

4.6 — Alertes et échéances

4.7 — Permanence

4.8 — Imports : AIR & greffe

4.9 — Documents et analyse OCR

4.10 — Sauvegarde et synchronisation

4.11 — Administration et rôles

5 Pile technique

6 Architecture détaillée

7 Sécurité et confidentialité

8 Déploiement et exploitation

9 Pour aller plus loin

1. En bref

APP MÉTIER est une **application de bureau Windows** (Electron + Next.js) développée pour le Parquet du Tribunal Judiciaire d'Amiens. Elle outille le quotidien des magistrats et des juristes assistants pour le suivi des enquêtes pénales sur trois contentieux spécialisés :

• **CRIMORG** Criminalité organisée • **ECOFI** Économique & financier • **ENVIRO** Environnement

Cas d'usage type. Un magistrat ouvre une nouvelle enquête, y rattache un ou plusieurs mis en cause, suit les actes (géolocalisation, écoute, saisies), reçoit des alertes avant les échéances de prolongation, consulte la cartographie relationnelle des dossiers du contentieux, génère des statistiques d'audience, puis archive le dossier. Les données sont synchronisées en temps quasi-réel entre les postes via un partage réseau, avec gestion fine des permissions par rôle et par contentieux.

Chiffres clés du projet

Lignes de code	≈ 47 000 (TypeScript / TSX)
Composants UI	≈ 133 fichiers React
Hooks personnalisés	25+
Modules utilitaires	30+
Handlers IPC Electron	150+ méthodes exposées au renderer
Cible	Windows portable (exécutable autonome)

2. Contexte et public visé

Le besoin métier

Les magistrats du parquet jonglent quotidiennement avec un grand nombre de procédures, chacune ponctuée d'**échéances légales strictes** (prolongations d'actes, durées maximales de détention provisoire, délais JLD). Les outils nationaux existants ne couvrent pas finement le suivi opérationnel par contentieux spécialisé ni la vision transversale entre dossiers. APP MÉTIER vient combler ce vide en offrant :

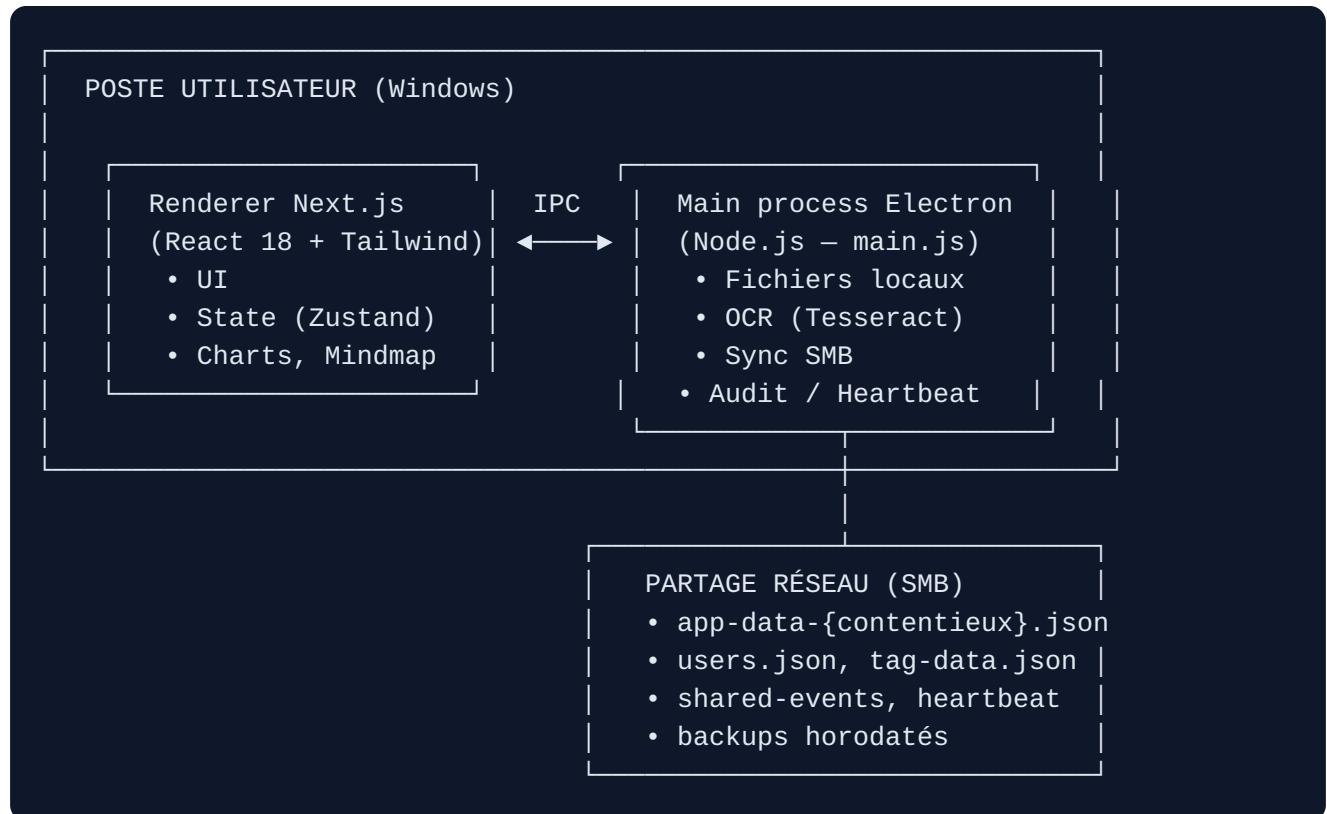
- Un **poste de pilotage unique** pour les enquêtes en cours et les instructions ouvertes.
- Des **alertes paramétrables** sur les délais critiques propres à chaque contentieux.
- Une **vision transversale** (cartographie) des liens entre enquêtes et personnes.
- Un **partage multi-utilisateur fiable** via le réseau interne, sans serveur applicatif central.

Profils utilisateurs

Rôle	Périmètre	Capacités principales
Administrateur	Global	Gestion des utilisateurs, paramétrage serveur, validation des mises à jour.
PRA / Vice-procureur	Global	Vision et édition tous contentieux, gestion alertes globales.
Magistrat	Par contentieux	Création, édition, archivage d'enquêtes et d'instructions sur ses contentieux.
JA (juriste assistant)	Par contentieux	Saisie, suivi, contribution sur les contentieux affectés.
JLD	Global, lecture seule	Consultation des dossiers, suivi des décisions à valider.

3. Architecture en un coup d'œil

APP MÉTIER est une application **Electron** qui embarque une application **Next.js** côté interface. Les données vivent en fichiers JSON locaux et sont synchronisées vers un **partage réseau** (SMB) accessible à tous les utilisateurs autorisés.



Aucun serveur applicatif n'est requis : la synchronisation repose sur des écritures atomiques sur un partage Windows partagé, avec gestion des conflits, journal d'événements partagés et présence (*heartbeat*) des utilisateurs.

4. Tour des fonctionnalités

4.1 — Tableau de bord et navigation multi-contentieux

La fenêtre principale présente, à gauche, une **barre latérale verticale** qui regroupe la navigation par contentieux (chacun avec sa couleur identifiante). Sous chaque contentieux apparaissent les vues *Enquêtes*, *Archives* et *Statistiques*. Les modules transverses (Instructions, Cartographie, Permanence, Alertes, Tags, Sauvegarde) sont accessibles en bas de la barre.

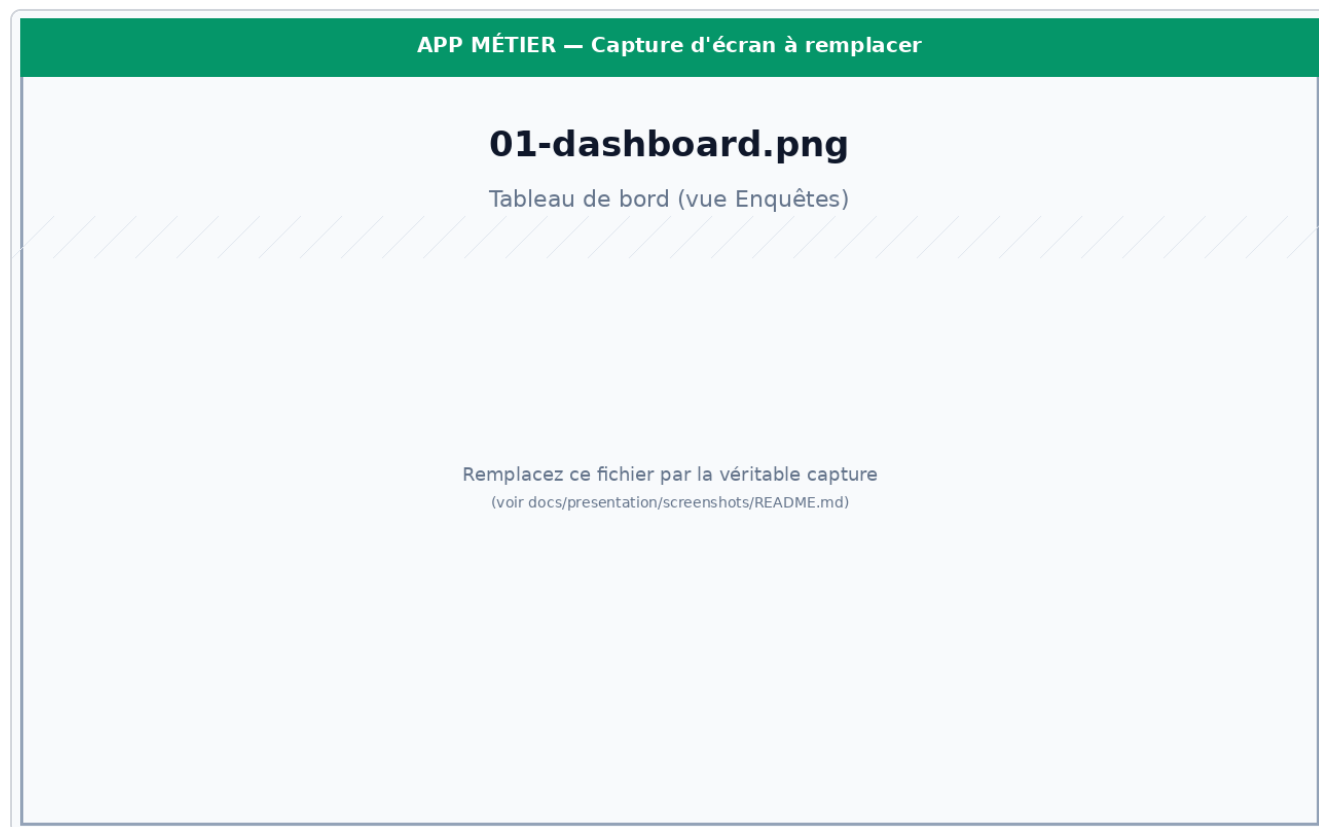


Figure 1 — Tableau de bord, vue Enquêtes du contentieux CRIMORG.

Le **header supérieur** contient la recherche globale (qui scanne aussi le contenu des documents indexés par OCR), un indicateur de synchronisation, un bouton de sauvegarde manuelle, l'indicateur d'état réseau, et la cloche d'alertes.

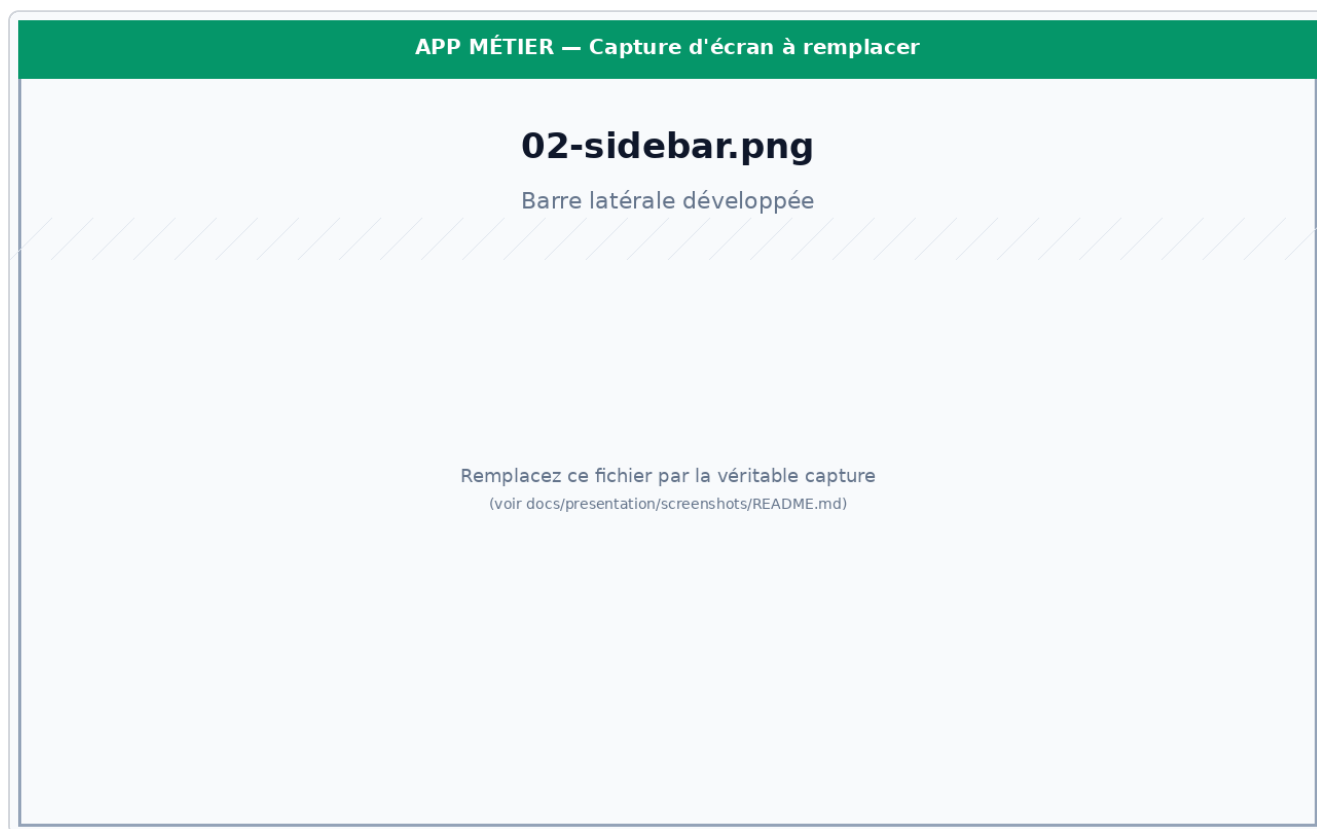


Figure 2 — Barre latérale avec les trois contentieux, badges d'alertes et lecture seule.

4.2 — Enquêtes : cycle de vie complet

Une enquête est représentée sous forme d'une **carte** dans la grille principale. Filtres et tri sont disponibles en barre supérieure (par service, infraction, durée, statut, présence de JLD, tags).

03-enquete-detail.png

Modale de détail d'une enquête

Remplacez ce fichier par la véritable capture
(voir docs/presentation/screenshots/README.md)

Figure 3 — Modale de détail d'une enquête, vue d'ensemble.

L'ouverture d'une enquête déploie une **modale structurée en 13 sections** :

- **En-tête** : numéro, dates, statut, contentieux, tags, services rattachés.
- **Mis en cause** : identité, alias, B1, ressortissants, mineurs.
- **Géolocalisation** : périodes, autorisations, prolongations.
- **Écoutes** : lignes interceptées, périodes, autorité de prescription.
- **Actes divers** (perquisitions, sonorisations, captations, etc.).
- **Saisies** : véhicules, immobilier, comptes, cryptoactifs, stupéfiants.
- **Co-saisine** : autres contentieux ou unités impliqués.
- **To-do** : check-list opérationnelle.
- **Transfert contentieux** : historique des déplacements de dossier.
- **Compte rendu** : rédaction libre, export DOCX possible.
- **Documents** : pièces jointes, OCR, catégorisation automatique.
- **Audience et Résultat d'audience**.

04-enquete-actes.png

Section Actes d'une enquête

Remplacez ce fichier par la véritable capture
(voir docs/presentation/screenshots/README.md)

Figure 4 — Section "Actes" : géolocalisation, écoute, autres actes, prolongations.

4.3 — Instructions judiciaires

Le module **Instructions** couvre les dossiers ouverts à l'instruction (post-information judiciaire). Il modélise :

- Les **cabinets** et magistrats / juristes assistants affectés.
- Les **mis en examen** avec leurs *infractions reprochées*.
- Une **timeline d'événements** (interpellation, DP, DML, débats JLD...).
- Les **mesures de sûreté** et leurs prolongations (durée légale et alertes 175 jours).
- Les **résultats** de vérifications et contrôles de légalité.

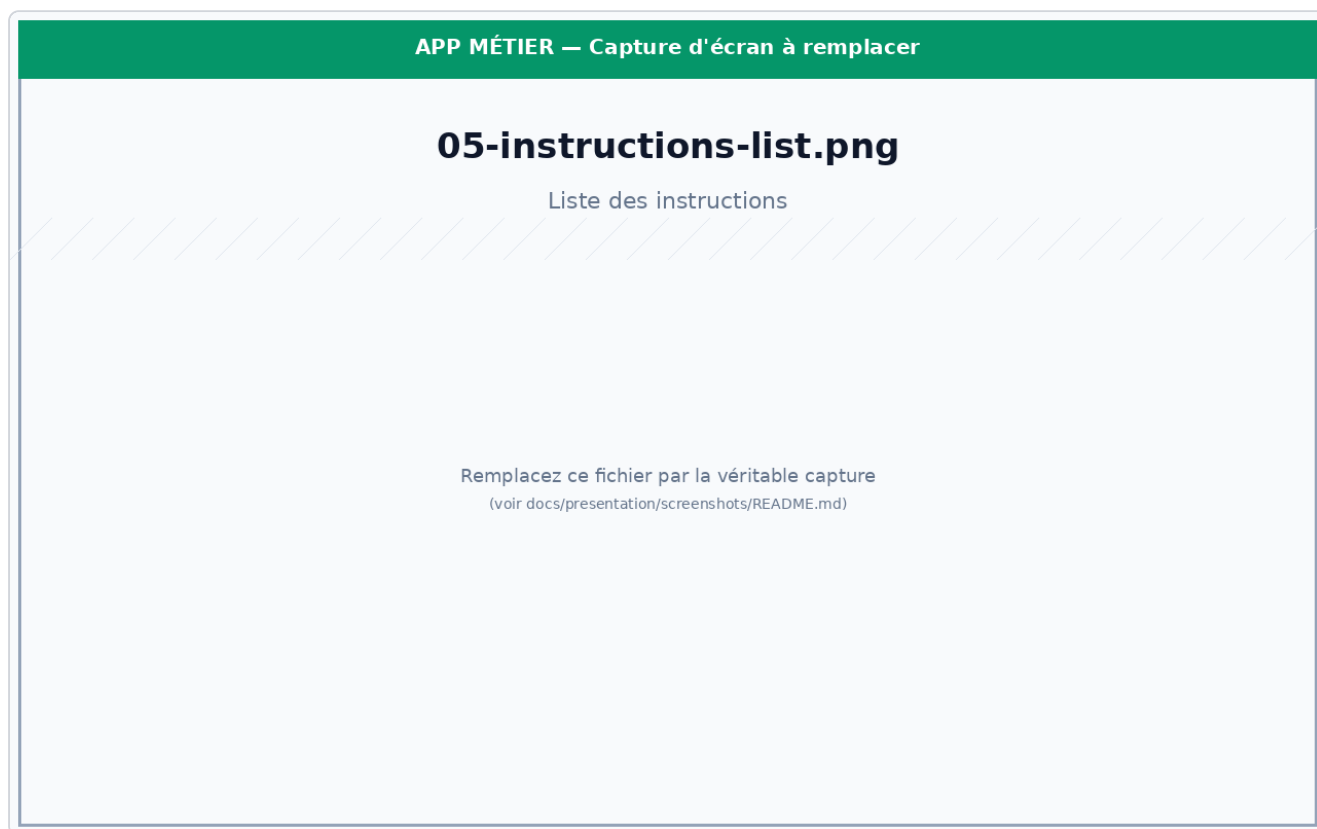


Figure 5 — Liste des dossiers d'instruction avec filtres par cabinet.

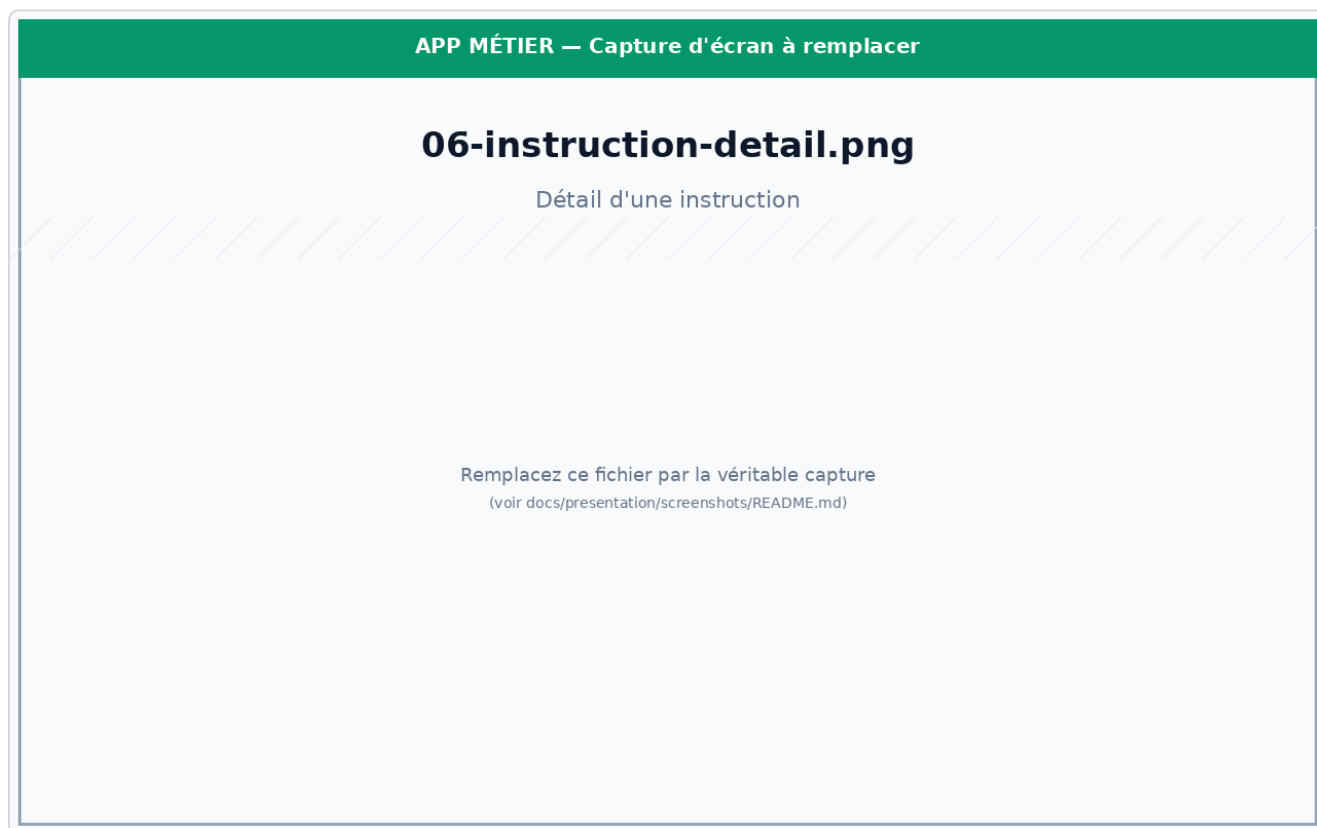


Figure 6 — Détail d'une instruction : mis en examen, timeline, mesures.

4.4 — Cartographie relationnelle

La **cartographie** (mindmap) propose une vue *force-directed* des dossiers d'un contentieux. Chaque nœud représente un mis en cause ; les liens matérialisent les co-occurrences entre enquêtes. Les nœuds sont regroupés en *galaxies* par type d'infraction et peuvent être enrichis d'*overlays* (focus CRIM ORG, influence convex hull). Un système de score paramétrable (degré, centralité, fréquence, présence cross-contentieux) hiérarchise visuellement les acteurs clés.

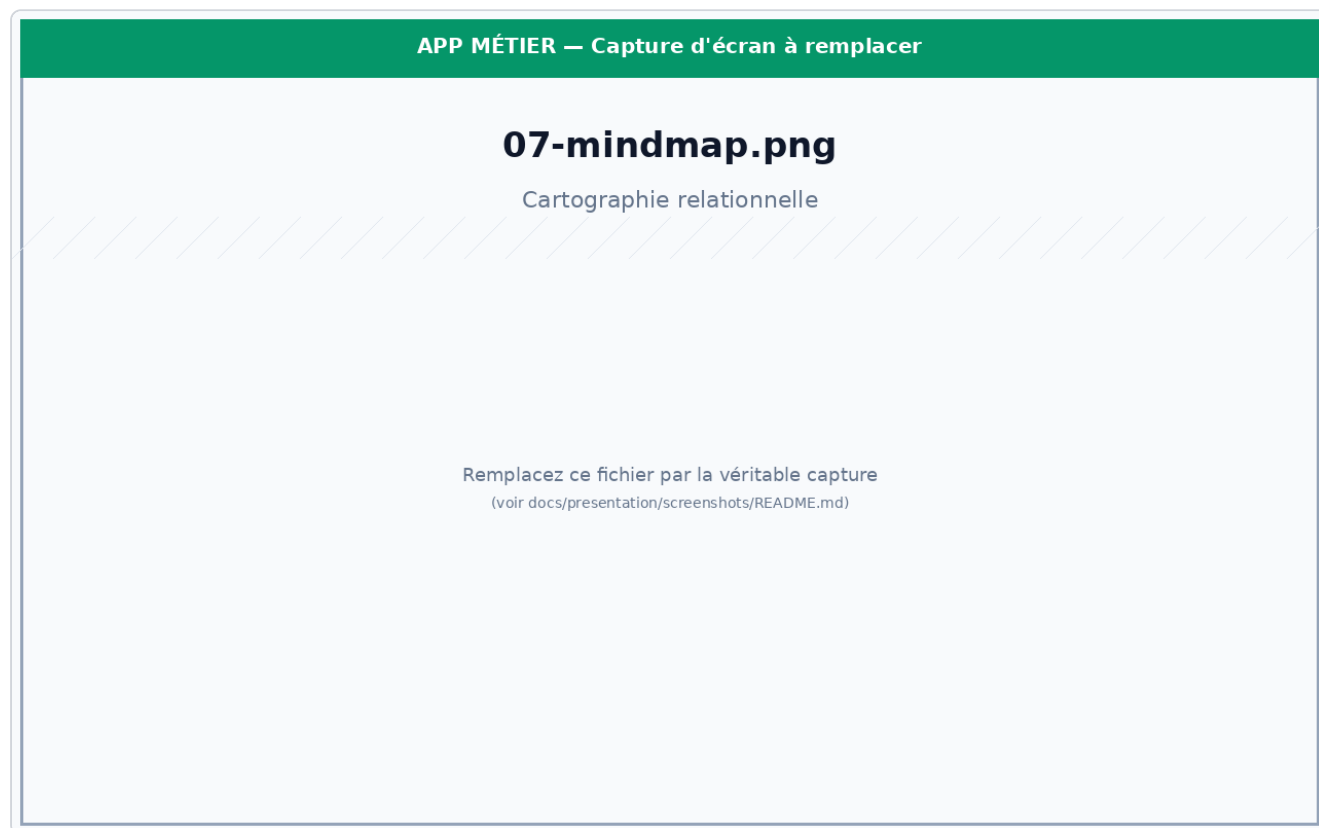


Figure 7 — Cartographie relationnelle : graphe interactif avec overlays.

4.5 — Statistiques

Trois tableaux de bord statistiques sont disponibles :

- **Statistiques générales** : volumes d'enquêtes, statuts, ventilations par service et infraction.
- **Statistiques d'audience** : type de poursuite (CRPC, CI, COPJ, OI, CDD), peines prononcées, saisies décidées.
- **Statistiques d'infraction** : répartition par catégorie, évolution temporelle.

Les graphiques sont exportables en PDF ou intégrés à un rapport via le bouton dédié (utilise `html2pdf.js` et `pdf-lib`).

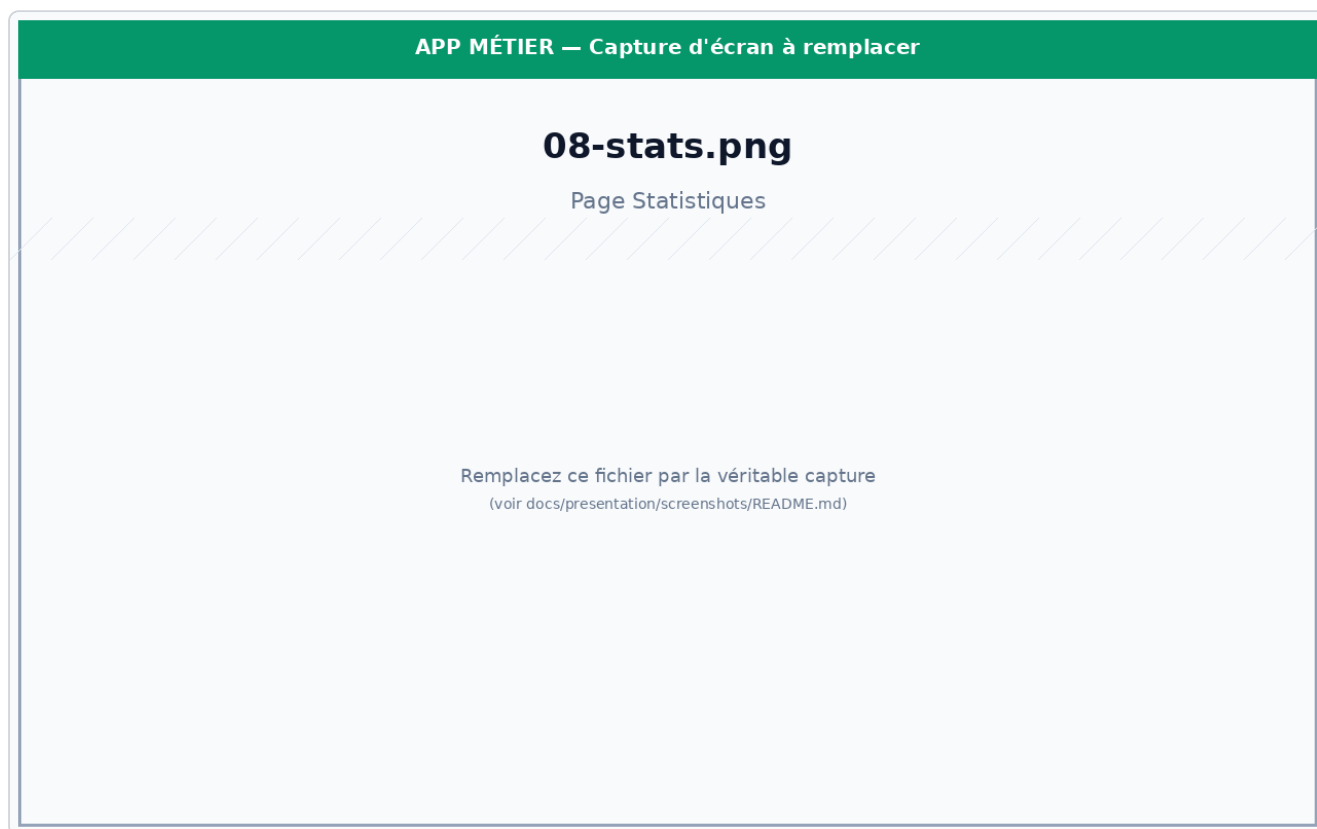


Figure 8 — Page Statistiques avec graphiques audience & infractions.

4.6 — Alertes et échéances

Le moteur d'alertes (`utils/alerts/alertManager.ts`) évalue en continu les **règles configurables** par contentieux :

- Échéance de prolongation d'acte (géoloc, écoute, autres).
- Délai de présentation au JLD.
- Plafonds de durée pour la détention provisoire (DP, DML, seuil 175 jours).
- Suivi des audiences en attente.

Chaque utilisateur peut affiner les seuils. Les alertes apparaissent sous forme de **badges** dans la barre latérale, d'une **cloche** dans le header, et de *toasts*. Une page dédiée centralise la consultation et la gestion des règles.

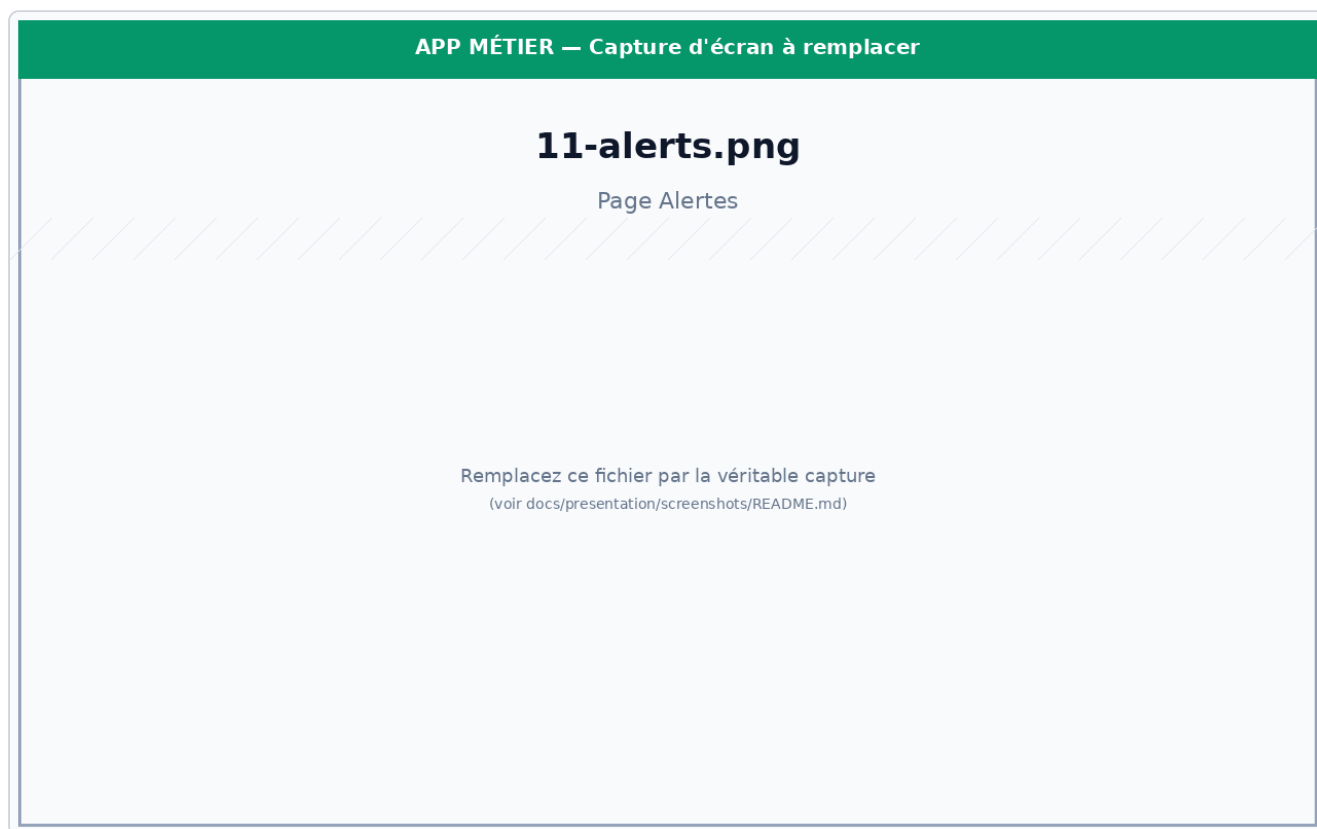


Figure 9 — Configuration des règles d'alerte et alertes actives.

4.7 — Permanence

La page **Permanence** affiche la grille hebdomadaire des magistrats et JLD de garde, filtrable et exportable en PDF. Elle s'appuie sur les préférences utilisateur synchronisées sur le serveur (voir §4.10).

10-permanence.png

Grille Permanence hebdo

Remplacez ce fichier par la véritable capture
(voir docs/presentation/screenshots/README.md)

Figure 10 — Grille hebdomadaire des permanences.

4.8 — Imports : AIR & greffe

Import AIR

Le module **AIR** permet l'import en masse depuis un fichier métier (CSV/Excel). Le flux comprend la sélection du fichier, le mapping automatique, la *vérification de doublons* et la création en lot.

Import greffe

L'import **greffe** rapatrie les éléments enregistrés au registre du tribunal pour les intégrer comme champs structurés dans les enquêtes (numéro de parquet, dates, parties).

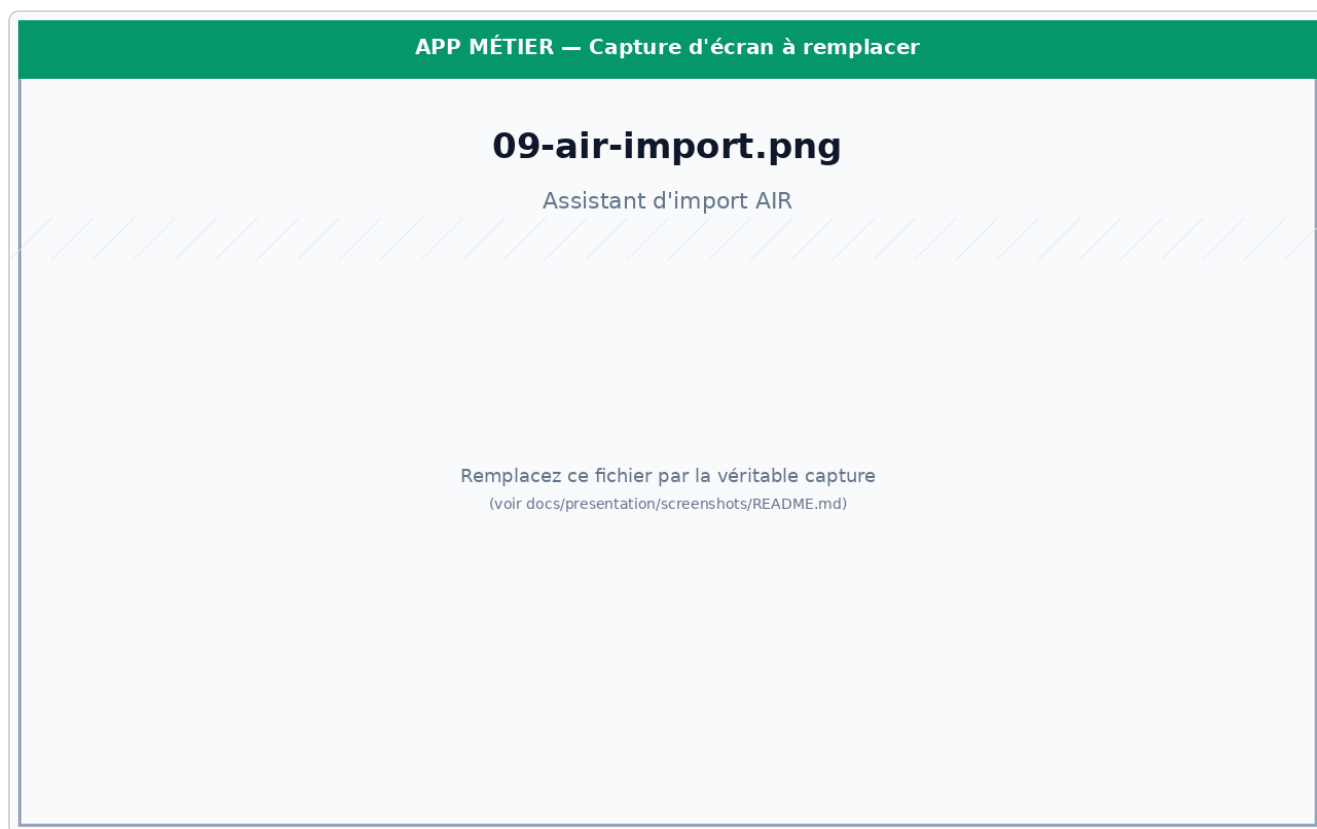


Figure 11 — Assistant d'import AIR, étape de vérification des doublons.

4.9 — Documents et analyse OCR

Chaque enquête peut accueillir des **pièces jointes** (PDF, DOCX, MSG, images). L'application :

- Stocke les documents en local (`data/documentenquete`) et/ou sur un chemin externe.
- Effectue une **analyse OCR** via *Tesseract.js* (données embarquées dans `tessdata/`).
- Extrait les métadonnées (dates, numéros, tribunal) et **catégorise automatiquement** (prolongation d'écoute, nouvelle écoute, autorisation, etc.).
- Permet la **recherche plein texte** dans le contenu indexé (recherche globale du header).

4.10 — Sauvegarde et synchronisation

Le module **Sauvegarde** est le poste de contrôle du multi-utilisateur. Il affiche :

- L'état du réseau (sain / lent / inaccessible) — sonde SMB toutes les 20 s.
- L'historique des sauvegardes horodatées (rotation paramétrable).
- L'outil de comparaison & restauration depuis une sauvegarde.
- Le suivi des conflits éventuels lors d'écritures simultanées.

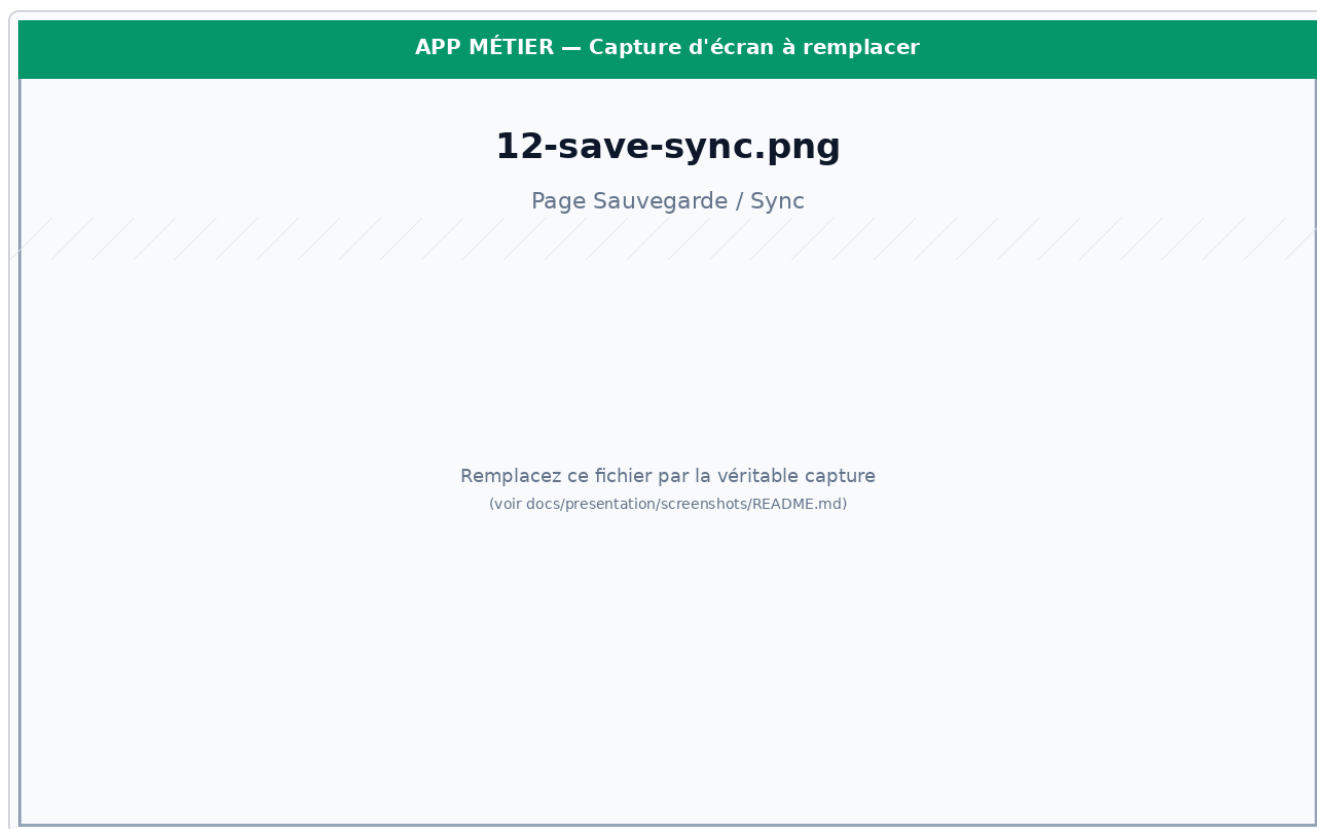


Figure 12 — Page Sauvegarde / Synchronisation.

La synchronisation est **par contentieux** et orchestrée par **MultiSyncManager**. Chaque écriture déclenche également un *shared event* côté serveur permettant aux autres postes d'être notifiés des changements pertinents (création, modification, archivage).

4.11 — Administration et rôles

Les administrateurs disposent de panneaux dédiés :

- **Utilisateurs** : création, validation, attribution des rôles globaux et par contentieux.
- **Contentieux** : chemins serveur, ordre d'affichage, libellés.
- **Cabinets & types d'événements** : configuration du module Instructions.
- **Scoring cartographie** : poids des critères de centralité.
- **Historique des tags** : audit des modifications.
- **Mises à jour** : validation centralisée avant déploiement sur les postes.

13-settings.png

Panneau Administration

Remplacez ce fichier par la véritable capture
(voir docs/presentation/screenshots/README.md)

Figure 13 — Panneau d'administration (utilisateurs).

5. Pile technique

Couche	Technologies	Rôle
Application desktop	Electron 30, Node.js (Windows portable)	Conteneur, accès système, IPC vers le rendu.
Interface	Next.js 13.5 (app router), React 18, TypeScript 5	Rendu de l'UI, navigation, lazy loading des modules lourds.
Styles	Tailwind CSS, Radix UI (15+ primitives), lucide-react	Design system cohérent, composants accessibles, icônes.
État	Zustand (6 stores), Jotai, React Context	État partagé performant, séparation par domaine.
Formulaires & validation	react-hook-form, zod	Saisie performante, validation par schéma.
Graphiques	Chart.js + react-chartjs-2, Recharts	Tableaux de bord statistiques.
Cartographie	@xyflow/react, d3-force	Graphe interactif, layout force-directed.
Documents	pdf-lib, pdfjs-dist, html2pdf.js, html-docx-js, xlsx	Export PDF/DOCX/XLSX, lecture de PDF.
OCR	tesseract.js (données tessdata/ embarquées)	Extraction de texte depuis images et PDFs scannés.
Persistance & sync	Fichiers JSON locaux, partage SMB Windows	Multi-utilisateur sans serveur applicatif.

6. Architecture détaillée

Processus Electron principal (`main.js`)

~3 300 lignes. Responsable de l'ensemble des accès système, exposés au renderer via un **context bridge** sécurisé (`preload.js`). Catégories de handlers IPC :

- **Stockage local** : `getData` , `setData` , écritures atomiques par contentieux, gestion des dossiers `data/` , `casiers/` , `documentenquete/` , `backups/` .
- **Documents** : copie vers chemin externe, scan, catégorisation, OCR.
- **Synchronisation** : `dataSync:pull/push` par contentieux (MultiSyncManager), `globalSync:*` pour les ressources globales (tags, alertes, audience, IDs supprimés, cartographie, préférences utilisateur).
- **Présence et événements** : `heartbeat:write/readAll` , `sharedEvent:write/readRecent` .

- **Audit** : `auditLog.append` pour la traçabilité des modifications.
- **Mises à jour** : `app:checkUpdate/applyUpdate` avec validation administrateur centralisée.
- **Résilience** : sonde réseau périodique, gestion des reconnections, reprise après veille (`powerMonitor`).

Pont de pré-chargement (`preload.js`)

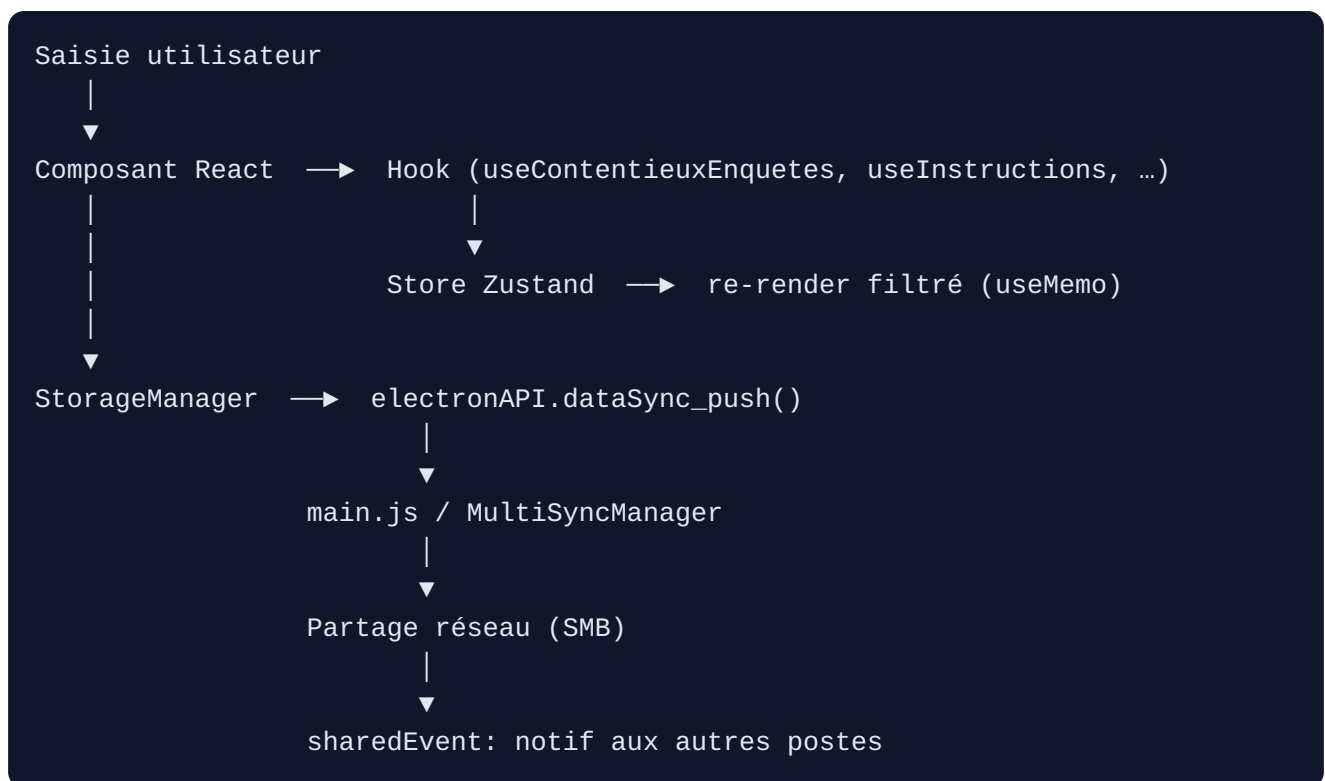
Isolation de contexte stricte : aucune API Node n'est exposée à la page rendue. Seules les méthodes énumérées sur `window.electronAPI` sont disponibles.

Render Next.js

Architecture orientée modules. Points d'entrée :

- `app/layout.tsx` : layout racine, polices, styles globaux.
- `app/page.tsx` : composition principale (sidebar + header + vue active).
- **Pages dynamiques** chargées en lazy : *AIRPage*, *InstructionsPage*, *InstructionArchivesPage*, *MindmapPage* — pour limiter le poids initial.
- **Contextes** : *UserContext* (auth, rôles), *ToastContext*, *AudienceContext*, *InstructionResultatsContext*.

Flux de données type



Modèle de données (extraits)

Domaine	Type principal	Champs clés
Enquête	Enquete	numéro, dates, statut, misEnCause[], geoloc, ecoutes, actes, saisies, tags, todos, documents, modifications[]
Instruction	DossierInstruction	cabinet, magistrat, misEnExamen[], infractions[], timeline[], mesures[], resultats[]
Audience	ResultatAudience	type (CRPC-Def, CI, COPJ, OI, CDD), peine, saisies décidées
Utilisateur	UserProfile	roles globaux, roles par contentieux, préférences
Cartographie	CartographieConfig	poids scoring (degré, centralité, fréquence, cross-contentieux)

Migrations de schéma

Le dossier `migrations/` définit les versions de schéma (`v1` , `v2`) et un `MigrationManager` qui applique les transformations au chargement si la version locale est antérieure. Garantit la compatibilité ascendante des données existantes lors des évolutions.

7. Sécurité et confidentialité

Données sensibles. L'application manipule des informations judiciaires (identités, infractions, mesures de sûreté). Toutes les couches intègrent des dispositifs de cloisonnement et de traçabilité.

Cloisonnement

- **Isolation de contexte Electron** : aucun accès Node depuis la page rendue ; toute interaction passe par le bridge typé du `preload.js`.
- **RBAC fin** : matrice de permissions par couple (*rôle, contentieux*). Le mode *lecture seule* est appliqué automatiquement pour les rôles JLD/JA sur les contentieux non écritures.
- **Chemins serveurs** définis par l'administrateur, vérifiés au lancement.

Traçabilité

- **Audit log** : toute modification (enquête, instruction, tags) consignée avec utilisateur, horodatage, type.
- **Modifications par dossier** : historique embarqué dans chaque enquête/instruction, consultable en interface.
- **Présence (heartbeat) et shared events** : permet de retracer qui agissait au moment d'un conflit.

Robustesse

- Désactivation des *source maps* en production (protection du code).
- Suppression de l'en-tête `X-Powered-By` (masquage de la stack).
- Écritures atomiques (fichier temporaire + renommage) pour éviter la corruption en cas d'interruption.
- Gestion explicite des `ECONNRESET` et reprises après veille.
- Rotation et conservation paramétrables des sauvegardes.

8. Déploiement et exploitation

Distribution

L'application est packagée en **exécutable Windows portable** (`electron-builder` cible `portable`). Les utilisateurs lancent l'application via `launcher.bat` qui détecte automatiquement Node.js portable et démarre Electron. Aucune installation système n'est nécessaire.

Première configuration

1. L'administrateur définit les chemins du partage réseau pour chaque contentieux.

2. Création du premier compte administrateur, puis ajout des utilisateurs et de leurs rôles.
3. Configuration des règles d'alertes par défaut.
4. Import initial (AIR/greffe) ou saisie manuelle des premiers dossiers.

Mises à jour

Un mécanisme de vérification des mises à jour interroge un dépôt distant (*GitHub releases*). Une fois une nouvelle version repérée, l'administrateur la **valide** pour qu'elle soit proposée aux autres postes — empêchant les déploiements non maîtrisés.

9. Pour aller plus loin

Pistes d'évolution envisagées

- Module de **reporting périodique** automatisé (génération d'un PDF mensuel synthétique).
- Ouverture aux **contentieux additionnels** (paramétrage purement configuration).
- Connecteur **messagerie chiffrée** pour le partage inter-juridictions.
- Amélioration de la **recherche transverse** avec un index plein texte persistant.

Ressources

Code source	Dépôt Git interne (branche de présentation : <code>c laude/app-presentation-docs-0818B</code>)
Captures d'écran	<code>docs/presentation/screenshots/</code>
Build PDF	<code>bash docs/presentation/build-pdf.sh</code>
Auteur	Audran Chevalier — Parquet du TJ d'Amiens

14-about.png

Écran À propos

Remplacez ce fichier par la véritable capture
(voir docs/presentation/screenshots/README.md)

Figure 14 — Écran "À propos" : auteur, version, mention légale.